WEB DESIGN • UNIT 07

Introduction to CSS

Learn to make HTML webpages readable, consistent and visually useful.

Library and Information Studies + Records Management Systems

Learning outcomes

After this unit, you should be able to:

- Explain how CSS differs from HTML.
- Read rules with selectors, properties and values.
- Use inline, internal and external CSS.
- Apply colours, typography, borders and spacing.
- Style an information-service webpage with a shared stylesheet.

TAKEAWAY **CSS is a practical foundation for later layout design.**

HTML and CSS have different roles

HTML says what information means; CSS says how it appears.

- HTML creates headings, lists, links, forms and tables.
- CSS changes colours, text sizes, spacing, borders and backgrounds.

```
<h1>Library services</h1>

h1 { color: #146b68; }
```

TAKEAWAY **A good webpage uses meaningful HTML before visual styling.**

CONCEPT

Why CSS matters in information services

Visitors should find key information without struggling through the layout.

- Make opening hours and service notices stand out appropriately.
- Keep catalogue links visually recognisable.
- Use consistent styling on library and records-service pages.
- Improve legibility through readable text sizes and spacing.

TAKEAWAY **Design supports access to information; it is not decoration alone.**

CSS rules: the basic pattern

A rule starts with a selector, followed by declarations inside braces.

- Selector: chooses which HTML elements to style.
- Property: names the visual feature to change.
- Value: specifies the new setting.

CODE EXAMPLE

```
h1 {
  color: #155c5a;
  font-size: 32px;
}
```

TAKEAWAY **Read a declaration as “property: value;”.**

DIAGRAM

Dissect a CSS declaration

Each part contributes to the final appearance.

h1 → selector

color → property

#155c5a → value

{ … } → declaration block

TAKEAWAY Curly braces group declarations for the selected element.

CODE

CSS punctuation and comments

Small syntax details matter when the browser reads CSS.

- Colon (:) separates property from value.
- Semicolon (;) separates declarations.
- Braces { } enclose the declarations for one rule.
- Comments help explain code; they do not appear on the page.

CODE EXAMPLE

```
/* Main service headings */
h2 {
  color: teal;
  margin-bottom: 12px;
}
```

TAKEAWAY **If a style does not apply, check punctuation and spelling first.**

CONCEPT

Three ways to add CSS

A style can belong to a single element, one webpage, or many pages.

- Inline CSS: one element has a style attribute.
- Internal CSS: rules appear inside a style element in HTML.
- External CSS: HTML links to a separate .css file.
- External stylesheets are usually most useful for a multi-page site.

TAKEAWAY **The same CSS rule can be reused without rewriting it for every element.**

CODE

Inline CSS: a quick example

Inline style attributes apply directly to an HTML element.

- Useful to recognise when reading existing code.
- Becomes difficult to maintain when repeated across many elements.

CODE EXAMPLE

```
<p style="color: green;">
  Library opening hours
</p>
```

TAKEAWAY **For coursework, prefer an external stylesheet when possible.**

Internal CSS: style one webpage

A style element in the document head holds CSS rules.

- Rules apply to matching elements within that HTML document.
- Useful for a simple single-page demonstration.

CODE EXAMPLE

```
<head>
  <style>
    p { color: #155c5a; }
  </style>
</head>
```

TAKEAWAY **Internal CSS belongs in the HTML document, not in a .css file.**

External CSS: connect two files

A link element in the HTML head points to a stylesheet.

- Keep index.html and styles.css together at first.
- If styles.css is in css/, write href="css/styles.css".
- A wrong path is a common reason styles do not appear.

CODE EXAMPLE

```
<!-- index.html -->
<link rel="stylesheet"
href="styles.css">

/* styles.css */
h1 { color: teal; }
```

TAKEAWAY **A shared stylesheet can be linked by several HTML pages.**

DIAGRAM

A shared stylesheet across a website

One stylesheet can control the same visual language on several pages.

index.html → styles.css

catalogue.html → styles.css

services.html → styles.css

Change one stylesheet; the linked pages can share the update.

TAKEAWAY **A consistent interface helps visitors move between services.**

CODE

Element selectors

An element selector uses the HTML tag name without brackets.

- p targets every paragraph.
- h1 targets every level-one heading.
- Use this when all elements of one type need a common appearance.

CODE EXAMPLE

```
p {
  font-size: 16px;
  line-height: 1.6;
}
h1 { color: #155c5a; }
```

TAKEAWAY **Element selectors are broad and easy to understand.**

Class selectors

A class selector begins with a dot. The HTML uses class without the dot.

- Class names can be reused on different elements.
- A class such as notice describes a role.
- Use reusable classes for service cards, notices and action links.

CODE EXAMPLE

```
<p class="notice">Open until 5 pm</p>

.notice {
  background: #fff0cb;
  padding: 12px;
}
```

TAKEAWAY In HTML: class="notice". In CSS: .notice.

CODE

ID selectors and other selector patterns

An ID selector begins with a hash sign and targets one named element.

- IDs should be unique in a document.
- Use h1, h2 to style several element types together.
- Use nav a to target links inside a navigation region.

CODE EXAMPLE

```
#contact { background: #eaf4ef; }
h1, h2 { color: #155c5a; }
nav a { text-decoration:
underline; }
```

TAKEAWAY **Choose selectors that match the intended scope of the style.**

Element, class and ID selectors

Compare what each selector describes and where it is useful.

- p → all paragraphs.
- .notice → any elements labelled as notices.
- #contact → one element with that ID.

Element = broad

Class = reusable

ID = specific

TAKEAWAY **Start with simple element and class selectors for practical work.**

CONCEPT

What does “cascading” mean?

Several declarations may try to style the same element.

- The browser resolves competing declarations according to its cascade rules.
- Among equally important rules with equal specificity, the later one generally wins.
- A class selector is usually more specific than an element selector at the same cascade level.
- Inheritance can pass some text properties from parents to children.

TAKEAWAY **The full cascade has additional rules, but these principles explain many beginner examples.**

CODE

A basic cascade example

Compare the two rules applied to the same paragraph.

- Both rules select p elements.
- They have the same specificity and importance.
- The later green declaration normally wins.

CODE EXAMPLE

```
p { color: blue; }
p { color: green; }

/* Paragraph text appears green.
*/
```

TAKEAWAY **Source order matters when competing rules otherwise have equal precedence.**

A class can be more specific

The class and element rules both match this paragraph.

- The class .notice is more specific than p.
- The notice text appears green at the same cascade level.
- Avoid !important while learning; understand the selectors first.

CODE EXAMPLE

```
<p class="notice">Service
notice</p>

p { color: blue; }
.notice { color: green; }
```

TAKEAWAY **Check the browser result and compare which selector matches.**

Colours: text and background

Colours can distinguish headings, messages and sections.

- color controls foreground text.
- background-color controls an element background.
- Hexadecimal values begin with #, such as #155c5a.

CODE EXAMPLE

```
body {
  color: #20343e;
  background-color: #f5f8f7;
}
.notice { background-color:
#fff0cb; }
```

TAKEAWAY **Use meaningful words as well as colour for status labels.**

Readable and difficult colour choices

The best-looking colour is not always the easiest to read.

- High contrast supports readers with different visual needs.
- Avoid very pale text on pale backgrounds.
- Do not use colour as the only status indicator.

Accessible contrast

Clear text labels

Consistent link styling

TAKEAWAY A public information website should work for a broad audience.

CODE

Typography with CSS

Text styling creates a readable information hierarchy.

- font-family selects the typeface.
- font-size controls text size.
- line-height controls spacing between lines.
- font-weight adjusts the visual emphasis of text.

CODE EXAMPLE

```
body {
  font-family: Arial, sans-serif;
  font-size: 16px;
  line-height: 1.6;
}
h1 { font-size: 2rem; }
```

TAKEAWAY **Readable paragraphs are more important than decorative fonts.**

CODE

Links must remain recognisable

Visitors should be able to identify the text they can activate.

- Keep link colours visibly different from ordinary text.
- An underline is a clear indication of a hyperlink.
- Avoid hiding links through poor contrast.

CODE EXAMPLE

```
a {
  color: #136b68;
  text-decoration: underline;
}
a:hover { color: #0b4543; }
```

TAKEAWAY The CSS changes link appearance; the HTML href still determines its destination.

CONCEPT

Border, padding and margin

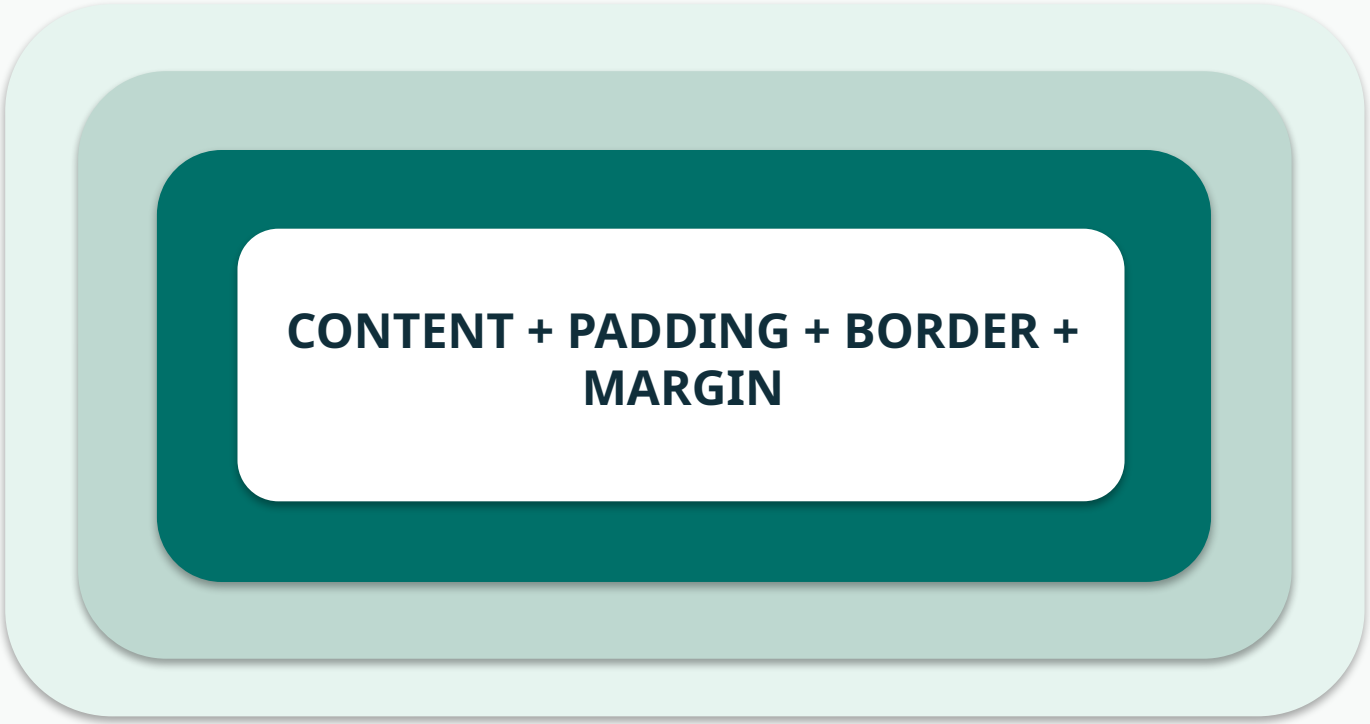
Spacing separates a service card from surrounding content.

- border is the visible line around an element.
- padding adds space between content and its border.
- margin adds space outside the border.
- border-radius softens a rectangular corner.

TAKEAWAY **More detailed box-model sizing appears in Unit 8.**

The box model in a service card

A box contains content, surrounding padding, a border and outside margin.



- Content: the words or controls.
- Padding: breathing space inside the border.
- Border: an optional visible edge.
- Margin: separation from adjacent elements.

TAKEAWAY **Avoid making information boxes so dense that text touches the edges.**

Use a reusable service-card class

One rule styles multiple information-service sections.

- Both cards share the class name service-card.
- A single CSS rule creates consistent spacing and borders.
- Changing one declaration updates all cards using the class.

CODE EXAMPLE

```
.service-card {
  background-color: #eff7f4;
  border: 1px solid #b7d7cc;
  padding: 16px;
  margin-bottom: 20px;
}
```

TAKEAWAY **Classes make a website easier to update as new services are added.**

Library versus records-centre examples

The CSS idea is identical; only the service content changes.

- Library: catalogue links, lending notices, reference services.
- Records: public access procedures, published guidance, enquiry options.
- Both require consistency, readable text and appropriate spacing.

Library site

Records centre

TAKEAWAY **Use invented information in practice webpages.**

Worked example: library homepage

A simple homepage can direct readers to services without excessive decoration.

- The header identifies the library and includes navigation.
- A notice highlights opening hours.
- Service cards introduce catalogue and reference services.
- The stylesheet gives cards shared spacing, borders and typography.

TAKEAWAY **A page is useful when visitors can find the next action quickly.**

CONCEPT

Worked example: public records centre

Public access guidance should be clear and distinguishable from restricted information.

- Place public procedures in clearly labelled sections.
- Use a notice for important access conditions.
- Use stable navigation and readable links.
- Do not treat CSS visibility or display settings as security controls.

TAKEAWAY **Authorisation and confidentiality require proper system controls.**

CODE

What if the stylesheet does not work?

Troubleshoot one issue at a time.

- Check the filename and href path.
- Look for misspelled selectors or properties.
- Check colon, semicolon and brace placement.
- Save the CSS file, then refresh the browser.

CODE EXAMPLE

```
<link rel="stylesheet"
href="styles.css">

/* File name must match exactly. */
.service-card { padding: 16px; }
```

TAKEAWAY **Inspect the browser preview after each small change.**

CONCEPT

Live CSS editor: experiment with a page

The lesson includes an editable CSS panel and a webpage preview.

- Choose Library or Records Centre.
- Change the main heading colour and text size.
- Change the page background and service-card padding.
- Select Update preview to compare results.

TAKEAWAY **The HTML content stays the same while the presentation changes.**

CONCEPT

Guided practical: external stylesheet

Use a two-file website to build confidence.

- Open index.html and examine the existing page.
- Open styles.css; change the page background.
- Adjust the h1 colour and the service-card padding.
- Save styles.css and refresh index.html.

TAKEAWAY **A useful CSS practical produces an observable change.**

CONCEPT

Assignment 7: style one webpage

Produce a simple styled information-service page.

- Use a fictional library or records-centre page.
- Link a styles.css file.
- Change the background and main heading appearance.
- Add a border and padding to one service section.
- Prepare the two files and one browser screenshot.

TAKEAWAY **No database or advanced JavaScript is required.**

CONCEPT

Review: six ideas to remember

A few well-understood CSS techniques are enough to style simple webpages.

- HTML structures content; CSS presents it.
- A rule = selector + declarations.
- External stylesheets help pages share a design.
- Classes are reusable; IDs identify individual elements.
- Text, colour, borders and spacing improve readability.
- Good CSS never replaces correct HTML or records security.

TAKEAWAY **Next: Page Layout and Responsive Design.**

UNIT 07 • COMPLETE

You can now style a webpage

Use your CSS knowledge to make information clearer.

- Try the Unit 7 practical.
- Complete the self-check quiz and review missed answers.
- Prepare the simple styled-page assignment.